Short Solutions for Large Rubik's Cubes and Squares

Eight Solvers on One Open Engine: Symmetry Portfolios that Beat the Santa 2023 Record Lengths, Net-Gain Bulk Commutators, and How Far from Optimal They Are

Gaurav
Shiv Nadar University
Greater Noida, India

Shree
Shiv Nadar University
Greater Noida, India

Sunaina
Shiv Nadar University
Greater Noida, India

Kashvi
Shiv Nadar University
Greater Noida, India

Shaurya
Shiv Nadar University
Greater Noida, India

Sharanya
Shiv Nadar University
Greater Noida, India

ABSTRACT

We study short solutions of the N×N×1 Rubik's Square and the N×N×N Rubik's Cube on one open engine, with eight solvers: our reproductions of IDA* with pattern databases, reduction with Kociemba's two-phase algorithm, Demaine et al.'s parallel cluster algorithm, the CayleyPy per-size network and, from its write-up, the KMCoders pipeline that produced the strongest public large-cube solutions in Kaggle's Santa 2023 competition; and three new methods. Our main method, Method 7, is a symmetry portfolio: the rotations and reflections of the puzzle turn one scramble into many equivalent inputs to a deterministic solver, and every answer maps back to a solution of the original state with the same length. Around KMCoders' own solver, with an optimal central 3×3×3 stage, it is the best method at 3 of the 8 Santa 2023 cube sizes, shorter in total than KMCoders' published best-of-many solutions on the puzzles it ran (4,522 against 4,620 quarter turns); around our strongest Square solvers it is the best method at 18 of the 18 Square sizes. Method 5C schedules Demaine's bulk commutators by net gain: a bulk move is applied to a whole block of layers when the pieces it fixes per move beat the best single-cluster fix. It is the strongest single solver on large Squares and, on random cubes, 26% shorter than Demaine's algorithm at 20×20×20, with a lead that grows with N. Method 5A, a cluster-token transformer trained on small Squares only, solves Squares of sizes it never saw. Finally, a counting lower bound, calibrated on Squares whose exact God's numbers we compute (6, 10, 12 for $N = 3, 4, 5$), shows how far everything is from optimal: the published Santa lengths are 1.81× the bound at $N = 4$ and 2.54× at $N = 33$, a gap that grows like $\log N$. We release the engine, the benchmarks, every solver and every result file.

CCS CONCEPTS

• **Computing methodologies** → **Search methodologies**; *Machine learning*; • **Theory of computation** → Mathematical optimization.

KEYWORDS

Rubik's Cube, heuristic search, parallel algorithms, commutators, beam search, pattern databases, macro-operators, size generalization, Cayley graphs

1 INTRODUCTION

The N×N×N Rubik's Cube is a standard testbed for planning on graphs that are far too large to enumerate. Its worst-case solution length, God's number, is 20 face turns for $N = 3$ [30] (26 quarter turns [29]) and grows as $\Theta(N^2/\log N)$ [9]; finding a shortest solution of a given state is NP-complete [10]. Learned solvers have recently become strong on small cubes. DeepCubeA [1] and EfficientCube [33] solve the 3×3×3, and CayleyPy [7] reaches near-optimal 3×3×3 solutions and the first learned 4×4×4 and 5×5×5 solvers. All of them train one network per size, and none reports a cube beyond $N = 5$. At the other end of the scale, the strongest public large-cube solutions come from a programming competition, Kaggle's Santa 2023 [17], where the team KMCoders solved, among others, a 33×33×33 in 4,719.0 quarter turns [19].

We ask what it takes to cover the whole range of sizes, for both the cube and its flat relative, the N×N×1 Rubik's Square of Demaine et al. [9]; which approach gives the shortest solutions at each size; whether any method can beat the published competition solutions; and how far all of them are from optimal. We implement eight solvers on one open engine: five reproductions of known methods (exact search, reduction, Demaine's parallel algorithm, CayleyPy and, from its write-up, the KMCoders pipeline) and three new ones, and we run them, together with three external baselines, on the same seeded random Squares and on the Santa 2023 cube puzzles. Section 2 lists what is new and what is reproduced.

Main results. Our main method is Method 7, a symmetry portfolio: the rotations and reflections of the puzzle turn one scramble into many equivalent inputs to a deterministic solver, and every answer maps back with the same length. (i) On the Santa 2023 cube puzzles, around KMCoders' own solver and with an optimal central 3×3×3, it is the best method at 3 of the 8 sizes, ahead of KMCoders' published best-of-many solutions (Table 5); on the puzzles it ran, its total is 4,522 quarter turns against 4,620 published (2.1% fewer; Table 9). (ii) On random Rubik's Squares, around our strongest solvers, it is the best method at 18 of the 18 sizes (Table 4). Our Method 5C, a net-gain schedule for Demaine's bulk commutators, is the strongest single solver on large Squares and, on random cubes, shorter than our reproduction of Demaine's algorithm by a margin that grows with N (26% on the 20×20×20), significant from $N = 5$ on with 20 states per size. (iii) A counting lower bound shows how much room is left: the published lengths are 1.81× the bound at

$N = 4$ and 2.54× at $N = 33$, a gap that grows like $\log N$ because the competition method fixes $O(1)$ pieces per 3-cycle in its final stage.

2 CONTRIBUTIONS

All contributions share one engine and one harness; every number in this paper is generated by script from the harness's result files. We separate what is new from what we reproduce.

Novel contributions (ours).

(1) **Method 7: a symmetry portfolio around deterministic solvers** (§7.8), our main method. The same scramble seen through the rotations and reflections of the puzzle (48 for the cube, 8 for the Square) is that many different inputs to a solver whose search is driven by fixed seeds and tie-breaks; each answer maps back to a solution of the original state with the same length, and Method 7 keeps the shortest. On the cube its inner solver is KMCoders' program with their beam variant, a 20× shorter annealing stage and, for odd N, an optimal quarter-turn 3×3×3 stage; on the Square, our strongest solvers.
(2) **Method 5C: net-gain scheduling of bulk commutators** (§7.6). It keeps Demaine's bulk moves but applies one to a whole block of layers when the pieces it fixes per move beat the best single-cluster fix, even if some cells of the block do not need it, scoring every candidate exactly.
(3) **Method 5A: one network for every size** (§7.5), a cluster-token transformer whose distance estimates carry over to Square sizes it never saw in training; with a staged beam search it solves every Square we tried and is second only to per-size networks up to $N = 7$.
(4) **How far from optimal** (§4.2): a counting lower bound on the typical optimal quarter-turn distance of every N×N×N, calibrated on the Squares whose exact God's numbers we compute, and measured against every method, including the published competition solutions.
(5) **An open engine and benchmark** (§5) for both puzzle families and every N: moves from geometry, clusters with N-invariant signatures, a solvability checker, three metrics, a harness that re-applies every solution, an exact adapter for the Santa 2023 cube puzzles, and the first exact God's numbers we are aware of for the 4×4×1 and 5×5×1.

Reproductions (our implementations of known methods).

(1) Method 1: our reproduction of IDA* with Korf's pattern databases [23] (§7.1).
(2) Method 2: our reproduction of reduction with Kociemba's two-phase algorithm [21] (§7.2).
(3) Method 3: our reproduction of Demaine et al.'s parallel cluster algorithm [9], with the 3-cycle batching suggested by KMCoders for practical sizes (§7.3).
(4) Method 4: our reproduction of the CayleyPy recipe [7], per-size networks and wide beam search (§7.4).
(5) Method 6: our reproduction of the KMCoders pipeline [19] from its written description only (§7.7); comparing it with their code isolates what the write-up leaves out.

We also run three external baselines unchanged through our harness (§6): KMCoders' published solutions, one run of KMCoders' own program, and dwalton76's NxNxN solver [36].

3 RELATED WORK

The cube group and its diameter. The 3×3×3 group and its generalisations are classical objects of group theory [16, 32]. Upper bounds on the 3×3×3's God's number fell from Thistlethwaite's 52 face turns to 26 by Kunkle and Cooperman's disk-based search [24], and Rokicki, Kociemba, Davidson and Dethridge settled it at 20 face turns by solving all $2.2 \cdot 10^9$ cosets of a subgroup on donated computer time [30]; in the quarter-turn metric it is 26 [29]. For larger cubes the diameter is unknown. Demaine et al. [9] prove that it is $\Theta(N^2/\log N)$ for both the N×N×N and the N×N×1: the lower bound is a counting argument (there are $e^{\Theta(N^2)}$ states but only $\Theta(N)$ moves, so some state needs $\Omega(N^2/\log N)$ of them), and the matching upper bound solves $\Theta(\log N)$ clusters with each shared move sequence. Ellenberg [11] explains why the counting bound is the natural guess, and the CayleyPy authors compute growth functions of many Cayley graphs to conjecture diameters [6]. Demaine, Eisenstat and Rudoy show that deciding whether a N×N×N state can be solved in k moves is NP-complete [10]. We use the counting argument quantitatively (§4.2): with exact state counts and the exact growth of canonical quarter-turn words it gives a concrete bound for every N that we can put next to every method.

Optimal and near-optimal 3×3×3 solvers. Thistlethwaite's algorithm and Kociemba's two-phase algorithm [21] solve the 3×3×3 by descending through subgroups; two-phase returns about 20 face turns in milliseconds, and min2phase is a fast engineered implementation [5]. Korf found the first optimal solutions of random states with IDA* and pattern databases [23], building on Culberson and Schaeffer's pattern databases [8]. Rokicki's `nxopt` [28] refines this with coset pruning tables of up to terabytes, 16-way symmetry and inverse lookups, in the half-turn and the quarter-turn metric; Kociemba's Cube Explorer also contains a quarter-turn optimal solver [20]. Method 1 is our reproduction of Korf's solver, and Method 7 uses `nxopt` for the 3×3×3 stage of odd cubes.

Large-cube solvers. Human and computer solvers for big cubes use *reduction*: solve the centers, pair the edges, and finish as a 3×3×3 [36]. Korf's macro-operators [22] formalise the move sequences that change one part of the state and restore the rest; commutators $[A, B] = ABA'B'$ are the standard way to build them, and conjugates sCs' move their effect anywhere. Demaine et al.'s parallel algorithm batches clusters that are in the *same* configuration so that one sequence of slice moves solves all of them; at practical sizes a center cluster has about $3 \cdot 10^{15}$ configurations, so exact matches are rare, and Method 3 and Method 5C batch 3-cycles instead. The dwalton76 solver [36] is a mature open reduction solver with large lookup tables for $N \leq 7$ and a general path above; we run it as a baseline.

Kaggle Santa 2023 and KMCoders. The Santa 2023 competition [17] asked for short solutions of 398 permutation puzzles: cubes from 2×2×2 to 33×33×33, "wreaths" and "globes", each given as a start state, a goal state and a set of allowed moves, scored by the total number of moves (§4.1). KMCoders published their solution, their

final submission and their Rust code [19]. For cubes they solve the special parts (corners, and for odd N the central 3×3×3) optimally, make every remaining cluster an even permutation and roughly align it with simulated annealing over elementary moves [18], then insert short commutators and finally 3-cycles at *arbitrary* positions of the move sequence, choosing time and sequence greedily or by beam search; their code also applies bulk commutators of slice sets chosen by annealing. Their reported lengths are the best of many runs. We reproduce the pipeline from the write-up (Method 6), run their program unchanged as a baseline, and build Method 7 around it.

Learned solvers. DeepCubeA [1] learns a cost-to-go function by approximate value iteration and searches with batch weighted A*. EfficientCube [33] trains directly on random scrambles from the goal. Q* search [2] scores all children of a node in one network pass and handles large action spaces. CayleyPy [7] trains a residual MLP to predict the length of a random walk from the solved state and searches with very wide beams; it reports over 98% optimal 3×3×3 solutions and the first learned 4×4×4 and 5×5×5 solvers. All of these models are trained per puzzle size on the full sticker array. Method 4 is our reproduction of CayleyPy; Method 5A instead decompose the puzzle into clusters whose networks are shared by all sizes.

Search techniques we build on. Beam search [26] keeps the W best nodes per depth; Method 4 and Method 5A use it, as does KMCoders' final stage. Simulated annealing [18] is KMCoders' second stage and Method 6's. Large-neighbourhood search [31], which destroys and rebuilds part of a solution, is what we tried, without success, on top of KMCoders' greedy stage (§7.8). Method 7 belongs to the line of work on randomisation and portfolios: run times and solution qualities of search algorithms are heavy-tailed over random seeds, so restarts and portfolios of independent runs pay off [13–15]. The cube's symmetries are usually used to *reduce* search, by storing one representative per symmetry class in pattern databases and coset tables [28, 30]; Method 7 uses them the other way round, as a source of diverse but equivalent inputs to a solver whose own randomness is fixed.

Size generalisation and other uses of the cube. Networks that read a problem as a set or sequence of parts, such as pointer networks [35] and set transformers [25], can be applied to larger instances than they were trained on; Method 5A applies this idea to cubie clusters. The cube also serves as a benchmark for robotic manipulation [27] and for spatial reasoning in (multimodal) language models [3, 4]; those works use small cubes and do not target solution length at scale.

4 PROBLEM, DATA AND METRICS

Puzzles. An N×N×1 Rubik's Square is an $N \times N$ grid of cubies with a top and a bottom colour, plus side stickers on the border. Its $2N$ moves are 180° flips of one row or one column; each move is its own inverse, and flips of two rows (or two columns) commute. An N×N×N Rubik's Cube has $6N^2$ stickers and $3N$ slices; a quarter, half or three-quarter turn of one slice is one move in the half-turn metric, giving $9N$ moves. Given a reachable state, the task is to return a short move sequence that solves it. On our random benchmark a state is *solved* when every face shows one colour, in any reachable orientation (24 for the cube, 4 for the Square); on Santa 2023 the goal is one exact state (below).

Table 1: Cubie cluster types. Wings combine two sticker orbits of one layer. Counts are for one N; $k = \lfloor N/2 \rfloor - 1$.

Puzzle	Cluster type	Stickers	Count
Cube	corners	24	1
	middle edges	24	1 if N odd
	wing edges	48	k
	centers (+, ×, oblique)	24	$((N-2)^2 - [N \text{ odd}])/4$
	fixed centers	6	1 if N odd
Square	corner	16	1
	edge	12	$2k$
	interior	8	k^2
	edge-cross, cross, center	6, 4, 2	odd N only

Clusters. A cubie can only ever reach the positions in its own cluster, and moves never mix clusters. Table 1 lists the cluster types and how many of each an N has; our engine finds them automatically (§5). A 20×20×20 has 2,400 stickers, 9 wing clusters and 81 center clusters, and every one must be solved.

Metrics. We report solution lengths after canonicalisation (merging commuting same-axis moves, §5) in three metrics: the *half-turn metric* (HTM), where a quarter, half or three-quarter turn of one slice counts 1; the *quarter-turn metric* (QTM), where a half turn counts 2; and the *slice-turn metric* (STM), where a block of adjacent layers turned together counts 1. On the Square all moves are flips, so HTM = QTM. The Santa 2023 score of a cube solution is exactly its QTM: every allowed move is a quarter turn of one slice and costs 1. Random-state results are in HTM, Santa results in QTM. To compare sizes we divide lengths by $N^2/\ln N$, the growth rate of the diameter, or by the counting bound of §4.2.

4.1 Data

Random states (our benchmark). A random state is the end of a long random canonical walk from the solved state (at least 25× the number of moves and at least 400 moves), which approximates a uniform random reachable state. Every instance is regenerated from a seed and checked by the solvability checker, and every method sees the same instances. The main comparison uses 10 Squares per size for $N \le 12$, 5 for $13 \le N \le 20$, 2 cubes per size for $N \le 10$ and one each at $N = 15$ and 20; the statistics of §8.3 use 20 states per size for cubes $N = 3$–10 and every Square size.

Kaggle Santa 2023 (external benchmark). Santa 2023 – The Polytope Permutation Puzzle [17] was a Kaggle optimisation competition (December 2023 to January 2024). Each of its 398 puzzles is a start state, a goal state and a list of allowed moves given as sticker permutations; a submission lists one move sequence per puzzle, and the score is the total number of moves, so every quarter turn counts 1. The puzzles are cubes 2×2×2 to 10×10×10, 19×19×19 and 33×33×33, "wreaths" and "globes". The goal is one exact state: the cube must end in the given orientation, not any rotation of it. Some cube puzzles have other goal patterns (checkerboard faces,

Table 2: Calibrating the counting bound on the Rubik's Square: state counts and exact God's numbers (colour goal, any of the 4 solved orientations) from breadth-first search, and the counting bound for $|G|/4$ states.

N	states	bound	God's number	ratio
3	768	4	6	1.50
4	82,944	6	10	1.67
5	10,616,832	8	12	1.50

or every sticker its own label), and some allow a number of wrong stickers ("wildcards"). We use the 188 cube puzzles whose goal has one colour per face and no wildcards; Table 5 gives their number per size. Our adapter derives the map between Santa's sticker order and the engine from the move permutations themselves (each Santa move must equal exactly one engine move) and checks it by replaying KMCoders' published solutions, which all reach the exact goal in our engine. The puzzle files are downloaded on first use and not redistributed.

4.2 How far from optimal? A counting lower bound

Every state at distance d is the end point of at least one *canonical* move word of length d, so if $c(d)$ counts such words, the radius L around the goal satisfies $\sum_{d \le L} c(d) \ge |G|$, with $|G|$ the number of reachable states. For the cube in QTM, a canonical word is a sequence of blocks; a block turns a non-empty set of layers of one axis, each by $+1$, -1 or 2 quarter turns, and consecutive blocks use different axes. A block's cost generating function is $f(x) = (1+x)^{2N} - 1$ and the words have $G(x) = 1 + 3f/(1-2f)$, so $c(d)$ grows like b^d with $b = 1/x^*$, $2f(x^*) = 1$, i.e.

$$b = \left(\left(\tfrac{3}{2}\right)^{1/(2N)} - 1 \right)^{-1}, \qquad L \;\ge\; \frac{\ln |G|}{\ln b} - O(1). \tag{1}$$

We count $|G|$ as colour states cluster by cluster: $8!\,3^7$ for the corners, $12!\,2^{11}$ for the middle edges, $24!$ for each wing cluster and $24!/(4!)^6$ for each non-fixed center cluster (same-coloured center pieces are indistinguishable); parity couplings change $\log_2 |G|$ by about one bit per cluster and are ignored. An exact count of words agrees with Eq. (1) to within one move (unit tests). Because words are overcounted (different canonical words may reach the same state), the bound stays valid. For the Square, row flips commute with row flips and column flips with column flips, so canonical words alternate between non-empty sets of rows and of columns, $f(x) = (1+x)^N - 1$ and $b = 1/(2^{1/N} - 1)$. Table 2 calibrates the bound where the truth is known: on the Squares whose exact God's numbers we compute by breadth-first search, the true worst case is 1.5–1.7× the bound. Typical optimal distances lie between the bound and the worst case. In this sense the bound turns "how good is a solution" into a number for every size, including sizes where no optimal solver exists.

5 ENGINE AND BENCHMARK

All methods run on one engine [12]. Everything below is generic over cuboid dimensions (d_0, d_1, d_2), so the cube (N, N, N) and the Square $(N, N, 1)$ come from the same code.

Moves from geometry. Each sticker stores a doubled, centred cubie coordinate and an outward normal. A move rotates every sticker of one slab by an integer rotation matrix; looking up the rotated (position, normal) pair gives a permutation π, and applying the move is the gather $s \leftarrow s[\pi]$, $O(6N^2)$ time. A slab whose cross-section is not square, such as one row of the Square, only gets 180° turns, and an axis of length 1 gets none. Unit tests check that every move has order 4 (or 2) and that each move followed by its inverse is the identity, for every tested N.

Canonical form. Moves on the same axis commute. We canonicalise a sequence by merging each same-axis run per layer and sorting by layer, and we use the same order to prune search: after a move on axis a, layer ℓ, no move on axis a with layer $\le \ell$ is allowed. This removes immediate repeats and every redundant ordering of commuting moves, and it is the word model of the counting bound (§4.2).

Clusters with N-invariant signatures. Union-find over all move permutations gives the sticker orbits; orbits sharing cubies are merged into cubie clusters. Inside a cluster, stickers are sorted by face and by the signs and relative magnitudes of their in-face coordinates. This order does not depend on N, so the moves acting on a cluster induce the same local permutations at every N. We hash the local generator set and the solved pattern into a *signature*; tests confirm that, for example, every ×-center cluster of the 5×5×5, 7×7×7 and 9×9×9 has the same signature. A model trained for one cluster type therefore applies at any size.

Projections. Restricting attention to a subset of stickers gives a *projected puzzle*. When it is small we enumerate it by breadth-first search from the solved pattern; the resulting table gives exact distances for that subset and doubles as an exact membership test.

Solvability checker. A state is accepted only if it passes: global and per-cluster colour counts; membership in the exact projection table for every cluster small enough to enumerate (all Square clusters); for cubes, valid corner, middle-edge and wing pieces, corner twist $\equiv 0 \pmod 3$ and, for odd N after normalising orientation by the fixed centers, even middle-edge flip and equal corner and middle-edge permutation parity; and for the Square, joint consistency of each cross cluster with the border clusters it shares moves with. These are necessary conditions. Property tests confirm that the checker accepts scrambled states and rejects a single corner twist, a single middle-edge flip, a swap of two corners on odd cubes and wrong colour counts.

Harness. Every solver implements one interface and returns engine moves. The harness re-applies the moves, decides *solved* itself, and writes one row per instance with lengths in all three metrics, wall time, nodes, memory, seed, hardware and git commit. A crash or time-out counts as a failure and stays in the denominator. External programs (KMCoders' solver, dwalton76, `nxopt`) run as subprocesses behind the same interface: their answers are parsed

into engine moves (wide turns expanded into single-slice turns) and re-verified like any other method's.

Shared beam search. Methods 4, 5A and 5B share one beam search. Per step it expands the beam in chunks, scores children with the heuristic, and keeps the W best after removing duplicates by a random 64-bit linear hash (also across the last three steps). Only (score, hash, parent, action) is stored per child; the next beam is rebuilt from parents. Beam states are packed at 3 bits per sticker and can live on the device, in pinned host memory or on disk. The search accepts *macro actions*: fixed move sequences applied as one step and expanded into ordinary moves in the output. A heuristic that can score all children of a parent in one pass is used that way, as in Q* search [2]. One step of a width-W search with A actions costs $O(WA)$ heuristic evaluations and $O(WA \cdot N^2)$ sticker work.

Pure 3-cycles for any cluster. Methods 2, 3 and 5C need, for every center and wing cluster of every N, move sequences that 3-cycle three of its pieces and touch nothing else. We find them automatically. For centers we enumerate commutators $[A, B\,C\,B']$ with A and C inner-slice quarter turns on one axis and B an outer face turn; for wings, $[X, Y\,Z\,Y']$ with X an inner slice and Y, Z outer faces. A candidate is kept if, applied to the sticker permutation, it moves exactly three cubies of one cluster. A conjugate $s\,c\,s'$ of a pure 3-cycle c is again pure, so breadth-first search over setup moves s (up to four) extends a few base commutators to every one of the $24 \cdot 23 \cdot 22/3 = 4{,}048$ 3-cycles of a cluster, each with a short move sequence.

6 EXTERNAL BASELINES

These programs and results are not ours. We run them unchanged through our harness (§5), which re-verifies every solution and measures it in our metrics.

B1: KMCoders' published Santa 2023 solutions. The final submission of KMCoders [19], replayed in our engine. Each of its 188 standard-goal cube solutions reaches the exact goal. These are the best lengths KMCoders kept per puzzle over the competition (their repository stores per-puzzle best files), so they reflect many runs and much tuning; we treat them as the published reference numbers. They exist only for the Santa puzzles.

B2: KMCoders' program, one run. KMCoders' Rust cube solver (`solve_cube` from the same repository), built from source and run once per state through a wrapper that writes the state in their input format. Our input matches their own puzzle files byte for byte (unit test). Their program caches per-puzzle intermediate results and, for odd N, reads its central-3×3×3 solution from a cache file that came from an external optimal solver in the competition (and otherwise calls a third-party solver from a fixed path); we supply that stage with Kociemba's two-phase algorithm, the program's own fallback. With these defaults B2 is "what one run of the published code gives". A two-line patch (released with our code) lets the random seed and the length of their annealing stage be set by environment variables; the defaults are unchanged.

B3: dwalton76. The open NxNxN reduction solver of Walton [36], with its lookup tables (about 8 GB for $N \le 7$, the range we ran), on the Santa puzzles. Its wide turns are expanded into single-slice turns before measuring.

Components. Kociemba's two-phase algorithm [21] (the `kociemba` Python package) finishes the 3×3×3 stage of Methods 2, 3 and 5C and of B2 for odd N. Rokicki's `nxopt` [28], built for the quarter-turn metric with a 1.42 GB pruning table, gives optimal QTM solutions of that stage in Method 7; one long-lived process keeps the table loaded and answers positions in seconds to about a minute each.

7 METHODS

We describe the eight methods in the order of their numbers. Each subsection states where the method comes from, its steps, its pseudocode, and its cost in moves, time and memory. Table 3 summarises them.

7.1 Method 1: IDA* with pattern databases (our reproduction)

Origin. Korf's optimal 3×3×3 solver [23], with pattern databases [8]; for the Square, the same search with an exact per-cluster heuristic, and breadth-first search for God's numbers.

Steps.

(1) *Pattern databases (3×3×3).* Breadth-first search from the solved state over cubie coordinates gives exact distances for three abstractions: the 8 corners ($8! \cdot 3^7 = 88{,}179{,}840$ entries) and two sets of 6 edges ($12!/6! \cdot 2^6 = 42{,}577{,}920$ each). Their move tables are derived from the engine's sticker permutations, and a test checks that the two agree. Largest entries: 11 (corners) and 10 (edges) in HTM.
(2) *Heuristic.* $h(s)$ = max of the three lookups, admissible and consistent.
(3) *IDA*.* Depth-first iterations with threshold $\theta = h(s_0), \theta + 1, \ldots$; children are pruned by $g + h > \theta$ and by the canonical move order of §5. The search runs in Numba, parallel over the root's moves (about $1.2 \cdot 10^7$ nodes per second on four cores), in HTM or QTM.
(4) *Square.* The heuristic is the maximum over clusters of each cluster's exact distance (from its projection table), admissible because no solution fixes a cluster in fewer moves than it needs alone. Since any of the four solved orientations counts, IDA* runs to each and keeps the shortest.
(5) *God's numbers.* For $N \le 5$, breadth-first search from all four solved Squares visits every reachable state and gives the exact distance of each.

Cost. Building the 3×3×3 databases is one breadth-first search over each abstraction (minutes; under 200 MB at 4 bits per entry). IDA* expands $O(b^{\,d-\bar{h}})$ nodes, with $b \approx 13.3$ the canonical HTM branching factor and $\bar{h}$ the typical heuristic value; at $d \approx 18$ for a random 3×3×3 this is beyond four cores in 30 minutes. Breadth-first search over the 5×5×1 stores $1.06 \cdot 10^7$ states (about 6 GB in Python).

Table 3: The eight methods. "Repro.": our reproduction of a published method; "new": ours. Move growth is the length of the returned solutions as a function of N (proved for Demaine's construction, empirical otherwise); time is per random state on our CPUs at the largest size we ran (details in each subsection and §8).

#	Method	Origin	Puzzles, sizes	Moves vs N	Time per state (largest size)
1	IDA* + pattern databases	repro. of Korf [23]	Square $N \le 5$, 3×3×3 scrambles	optimal	seconds (5×5×1); random 3×3×3: none in 30 min
2	reduction + two-phase	repro. (standard)	both, $N \le 20$	$\Theta(N^2)$	seconds to minutes (20×20×20)
3	Demaine's parallel algorithm	repro. of [9]	both, $N \le 20$	$\Theta(N^2/\log N)$ (theory)	minutes (20×20×20)
4	CayleyPy per-size network	repro. of [7]	Square $N \le 8$	near-optimal where trained	seconds (8×8×1, W=4096)
5A	cluster-token transformer	new	Square $N \le 14$	$\Theta(N^2)$	1.5 min (14×14×1, T4)
5C	net-gain bulk commutators	new	both, $N \le 20$	between $N^2/\log N$ and N^2 (empirical)	2.6 min (20×20×20)
6	KMCoders pipeline from its write-up	repro. of [19]	cube, $N \le 19$ (Santa)	$\Theta(N^2)$	14 min (19×19×19)
7	symmetry portfolio (main method)	new	cube (Santa), Square $N \le 20$	as its inner solver	$k\times$ the inner solver; 17–28 CPU-min (4×4×4)

Algorithm 1: Method 1: IDA* (our reproduction of Korf's solver)

Input: state s_0; admissible heuristic h; canonical successor function
Output: an optimal move sequence

$\theta \leftarrow h(s_0)$;
while *true* **do**
  $(r, \mu) \leftarrow \textsc{Dfs}(s_0, g{=}0, \theta, [\,])$;
  if r = *found* **then return** μ;
  $\theta \leftarrow r$; `// smallest g + h that exceeded θ`
Function $\textsc{Dfs}(s, g, \theta, \mu)$**:**
  if $g + h(s) > \theta$ **then return** $(g + h(s), \cdot)$;
  if s *solved* **then return** (found, μ);
  $m \leftarrow \infty$;
  for *each canonical move a after* μ*'s last move* **do**
    $(r, \nu) \leftarrow \textsc{Dfs}(s \cdot a, g + 1, \theta, \mu \| a)$;
    if r = *found* **then return** (r, ν);
    $m \leftarrow \min(m, r)$;
  return $(m, \cdot)$;

Algorithm 2: Method 2: reduction (our reproduction), cube

Input: state s; per-cluster pure 3-cycle libraries $\mathcal{L}_c$
Output: move sequence μ

$\mu \leftarrow$ orientation moves;
for *each wing cluster* c *with odd permutation* **do** append one inner-slice quarter turn of c to μ;
$\mu \leftarrow \mu \,\|\, \textsc{Greedy3Cycles}$(center clusters);
$\mu \leftarrow \mu \,\|\, \textsc{TwoPhase}$(corners, middle edges, fixed centers);
$\mu \leftarrow \mu \,\|\, \textsc{Greedy3Cycles}$(wing clusters);
return canonicalise(μ);
Function $\textsc{Greedy3Cycles}(F)$**:**
  $\nu \leftarrow [\,]$;
  while *some cluster of* F *is unsolved* **do**
    $\beta \leftarrow \arg\max_{c \in F,\ \beta \in \mathcal{L}_c}$ (pieces of c fixed by β) / $|\beta|$;
    $s \leftarrow s \cdot \beta$; $\nu \leftarrow \nu \,\|\, \beta$;
  return ν;

7.2 Method 2: reduction with Kociemba's two-phase algorithm (our reproduction)

Origin. The standard reduction method for large cubes [36], with Kociemba's two-phase algorithm [21] for the final 3×3×3, and Demaine et al.'s lemmas for the Square.

Steps (cube). After relabelling colours so that the target is one fixed orientation (for odd N the fixed centers decide; for the exact Santa goal, middle-slice turns first bring the fixed centers home):

(1) *Wing parity.* Every wing cluster with an odd permutation gets one inner-slice quarter turn (Demaine's Lemma 7), since 3-cycles are even.
(2) *Centers.* Every center cluster is solved with its pure 3-cycles (§5), greedily: at each step the 3-cycle with the most pieces fixed per move.
(3) *3×3×3 stage.* Corners, middle edges and fixed centers behave as a 3×3×3 under outer-face turns and are solved by Kociemba's two-phase algorithm; for even N an odd corner permutation (PLL parity) is fixed by one outer quarter turn first.
(4) *Wings.* Every wing cluster is solved with its pure 3-cycles, greedily.

Outer-face turns keep center clusters on their faces, and pure 3-cycles keep everything else, so the stages never undo each other.

Steps (Square). The frame and the edge and cross clusters are solved exactly on their projections (Demaine's Lemma 3, $O(N)$ moves), then each interior cluster with its four commutators $c\,r\,c\,r$ (Lemma 2), at most three each.

Cost. A cube has $\Theta(N^2)$ center clusters of 24 pieces, each needing at most about 12 pure 3-cycles of 8–12 moves, so solutions have $\Theta(N^2)$ moves. Building the libraries is a breadth-first search over setups for each cluster type (seconds); each greedy step scans all clusters' libraries ($O(N^2 \cdot 4{,}048)$ lookups).

7.3 Method 3: Demaine's parallel cluster algorithm (our reproduction)

Origin. Demaine et al. [9]. On the Square we follow their construction; on the cube, where batching by identical configuration is useless at practical sizes, we batch 3-cycles, as the KMCoders write-up suggests [19].

Steps. Method 3 shares every stage with Method 2 except the batching.

Algorithm 3: Method 3: one family of cube clusters (our reproduction with 3-cycle batching)

Input: state s; type bank $\mathcal{T}$; libraries $\mathcal{L}_c$
Output: move sequence μ

while *some cluster unsolved* **do**
  for *each type* $t \in \mathcal{T}$ **do**
    $H_t \leftarrow$ the layer pairs (i, j) where type t fixes pieces of cluster (i, j) and harms none;
    cover H_t by blocks $I \times J \subseteq H_t$; score each by pieces fixed $/(2|I| + 2|J| + 4)$;
  $v \leftarrow$ best block, or the best single 3-cycle if that scores higher;
  $s \leftarrow s \cdot v$; $\mu \leftarrow \mu \| v$;
return μ;

(1) *Square.* Each interior cluster's optimal word of commutators is read off its exact table; round t applies the t-th commutator of every cluster, and the clusters needing the same commutator are covered by blocks of columns X and rows Y, each fixed by one bulk sequence $C_X R_Y C_X R_Y$ of $2|X| + 2|Y|$ moves (Lemma 4), with Demaine's column groups (Lemmas 5–6).
(2) *Cube, types.* A commutator *type* is an outer-face setup plus a template with layer parameters, such as $[A_i, B\, C_j\, B']$; with layer sets I and J (all slices of I turned together) it applies the same 3-cycle pattern to every cluster $(i, j) \in I \times J$ for $2|I| + 2|J| + 4$ moves.
(3) *Cube, rounds.* Each round picks the type with the most pieces fixed per move over all clusters and applies it to every block of clusters it helps, or applies one single-cluster 3-cycle when that is better.

Cost. Demaine's construction is $\Theta(N^2/\log N)$; our 3-cycle batching gives no such guarantee, and its blocks shrink as clusters diverge (§7.6). Each round scores $|\mathcal{T}|$ types over $O(N^2)$ layer pairs.

7.4 Method 4: CayleyPy per-size networks (our reproduction)

Origin. Chervov et al. [7].

Steps.

(1) *Data.* Fresh non-backtracking random walks from the solved state; each visited state is labelled with its walk length k (a diffusion distance).
(2) *Model.* One residual MLP per puzzle size on one-hot sticker inputs, trained with MSE to predict k.
(3) *Search.* Beam search with hash deduplication (§5), width W, at most 200 steps, scoring children by the network; optionally retried at $2W, 4W, \ldots$

Square models have 0.25–0.45M parameters and were trained on 4M ($N \le 5$) or 10M ($N = 6$–8) examples in 1–4 minutes each on CPU. A second set of Square models with the "large" preset (1.27M parameters at $N = 8$, 256M examples) was trained on a Colab T4 GPU (§8.2).

Cost. Training is $O(\text{examples} \times \text{parameters})$; search is $O(T\,W\,A)$ network evaluations with A moves ($2N$ for the Square). Each size needs its own model and training run.

Algorithm 4: Method 4: beam search with a learned distance (our reproduction of CayleyPy)

Input: state s_0; network $\hat{d}$; width W; step limit T
Output: move sequence or failure

$B \leftarrow \{s_0\}$;
for $t \leftarrow 1$ **to** T **do**
  $C \leftarrow \{s \cdot a : s \in B,\ a$ a move$\}$ minus states seen in the last 3 steps (by hash);
  if *some* $c \in C$ *is solved* **then return** its path;
  $B \leftarrow$ the W states of C with the smallest $\hat{d}$;
return failure;

7.5 Method 5A: one network for every size (new)

Idea. Replace the per-size sticker network of Method 4 with one that reads the puzzle as a set of cluster tokens, so the same weights apply at every N, and use its per-cluster costs to guide a staged search on sizes too large for full-puzzle beam search.

Steps.

(1) *Tokens.* Each cluster becomes one token: the one-hot colours of its stickers in the canonical order, projected by a linear map shared by all clusters of the same type, plus a learned embedding of the cluster kind and a small MLP of geometric features (the cluster's two in-face depths divided by $N - 1$, and $N/20$).
(2) *Encoder.* A transformer encoder [34] without positional order mixes the tokens, since they form a set [25].
(3) *Additive head.* Each token predicts a non-negative cost; the costs are summed and a small global correction is added. A sum matches the structure of the problem, since total distance grows with the number of unsolved clusters, and it extends to any number of tokens.
(4) *Training.* Fresh non-backtracking random walks on every training size ($N = 4, \ldots, 8$), target the walk length k, loss on $(\hat{k} - k)/(N^2/\ln N)$ so that all sizes contribute on one scale. The model in the main table is the GPU-trained one (width 192, four layers, six heads, 20,000 steps of 512 examples on a Colab T4).
(5) *Search, $N \le 7$.* Full-puzzle beam search (Algorithm 4 with this network).
(6) *Search, $N \ge 8$: staged beam search* (Figure 1, Algorithm 5). Following Demaine et al. [9], the Square is solved in stages from the outside in: the *frame* (corner, center and edge-cross clusters), then edge clusters two at a time, then cross and interior clusters three at a time. Each stage adds a few pieces to an exact goal: those pieces, and every piece solved earlier, must show their solved stickers, so later stages cannot undo earlier work. A stage searches only the moves that touch one of its targets plus the macro actions that do: for every row r and column c, the commutator $c\,r\,c\,r$, which changes only the cluster at their crossing (Demaine's Lemma 2), and the conjugates $r\,c\,r$ and $c\,r\,c$. The stage heuristic is the sum of Method 5A's per-token costs over the stage's target clusters and the earlier clusters its actions can move. If the projection of a stage's goal stickers has at most 10^5 states, it is enumerated and solved optimally by lookup (the frame

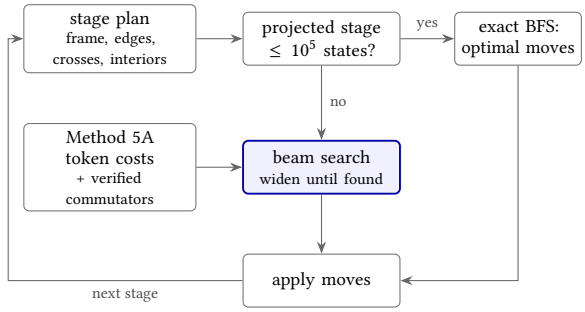


Figure 1: Method 5A's staged search for $N \geq 8$. Each stage adds a few pieces to an exact goal. Tiny stages are solved optimally by lookup; the rest by beam search over single moves and verified commutators, guided by Method 5A's per-cluster costs.

Algorithm 5: Staged beam search (Method 5A for $N \geq 8$)

Input: state s, stage plan $S_1, \ldots, S_m$, cost model h
Output: move sequence μ with $s \cdot \mu$ solved

$\mu \leftarrow [\,]$;
for $i \leftarrow 1$ **to** m **do**
  $G \leftarrow \bigcup_{j \leq i} \mathrm{goal}(S_j)$;
  if *s already solved on G* **then continue**;
  $A \leftarrow$ single moves and verified macros that move some target of S_i;
  if *projection of G under A has $\leq 10^5$ states* **then**
    $v \leftarrow$ optimal moves by greedy descent on its BFS table;
  else
    $W \leftarrow W_0$;
    repeat
      $v \leftarrow \textsc{BeamSearch}(s, A, h, W, \mathrm{goal} = G)$;
      $W \leftarrow 4W$;
    until *v found* **or** $W > W_{\max}$;
  if *no v* **then return** failure;
  $s \leftarrow s \cdot v$; $\mu \leftarrow \mu \,\|\, v$;
return canonicalise(μ);

always qualifies: for any N it is a 3×3×1). Each stage starts at width 128 and retries at 4× the width up to 2,048.

Cost. One evaluation is $O(K^2 d + K d^2)$ for $K = \Theta(N^2)$ cluster tokens and width d; a beam step costs $O(W \cdot |A|)$ evaluations. On a T4 a 10×10×1 takes about 9 s and a 14×14×1 about 90 s; on CPU several times longer. There are $\Theta(N^2)$ stages, so solutions have $\Theta(N^2)$ moves.

7.6 Method 5C: net-gain scheduling of bulk commutators (new)

Idea. Method 3 applies a commutator type to a block $X \times Y$ of layer pairs only when *every* cell of the block helps the cluster it acts on. Early on many clusters need the same 3-cycle and blocks are large, but the clusters soon diverge, the blocks shrink to single cells and batching fades: on our 20×20×20, 244 of Method 3's 292 center rounds fixed a single cluster. Method 5C changes only this schedule.

Algorithm 6: Method 5C schedule for one family of clusters (centers or wings)

Input: state s; type bank (verified pure 3-cycles per type, cluster and layers); per-cluster 3-cycle libraries
Output: move sequence μ solving every cluster of the family

$\mu \leftarrow [\,]$;
while *some cluster is unsolved* **do**
  $c_1 \leftarrow$ best single-cluster 3-cycle; $\lambda \leftarrow$ pieces fixed per move of c_1;
  for *each of the 24 types with the largest total positive gain* **do**
    $(X, Y) \leftarrow$ alternating maximisation of Eq. (2);
    apply the bulk commutator to s; discard it if anything outside the clusters moved;
    record its exact gain and its moves (less shared setup);
  pick the best gain per move among the blocks and c_1 (optionally after one step of lookahead);
  $s \leftarrow s \cdot v$; $\mu \leftarrow \mu \,\|\, v$;
return μ;

Steps.

(1) *Net-gain blocks.* For a type t, let $g_t(i, j)$ be the change in the number of solved pieces of the cluster that cell (i, j) acts on (it can be zero or negative), and λ the best number of pieces per move that any single-cluster 3-cycle offers in the current state. Method 5C chooses

$$(X^*, Y^*) = \arg\max_{X \cap Y = \emptyset} \sum_{(i,j) \in X \times Y} g_t(i, j) \;-\; \lambda \left(2|X| + 2|Y| + 4 + 2|s_t|\right), \tag{2}$$

where s_t is the type's setup and every cell of $X \times Y$ must be a verified pure 3-cycle. A block is worth applying exactly when its pieces per move beat the best single fix, so cells that do not help, or slightly hurt, are accepted when the block still pays. We maximise by alternation (rows given columns, columns given rows) from the three best rows.

(2) *Exact scoring.* Because several cells may act on one cluster, each candidate is *applied* to the state and scored exactly; this also checks that nothing outside the clusters moved. Wings have one layer parameter and the same rule with Y omitted.

(3) *Setup sharing.* A type whose setup equals the previous step's is charged $2|s_t|$ moves less, because the shared $s_t' s_t$ cancels when moves are canonicalised.

(4) *Lookahead.* With one step of lookahead (used for $N \leq 15$, where it is affordable), the four best candidates are re-scored by the best step available after them.

(5) *Everything else is Method 3.* Wing parity, the 3×3×3 stage, the Square frame and edge stages, the type bank and the single-cluster fallback are shared, so the difference between the two methods is the schedule alone. On the Square, the net-gain schedule of the interior clusters (gain = drop in exact distance) and Method 3's are both computed and the shorter is kept.

Cost. Each round scores 24 types, each by alternation over $O(N)$ rows and columns, plus exact application ($O(N^2)$ per candidate), so a round is $O(N^3)$ and the number of rounds is at most the number

of center pieces, $O(N^2)$; lookahead multiplies the time by about four. Measured: 2.6 minutes for a 20×20×20. Moves: bulk rounds fix many clusters per move, which is what moves Method 5C's ratio to $N^2/\ln N$ more slowly than the other methods' (§8.3); we have not proved a bound.

7.7 Method 6: the KMCoders pipeline from its write-up (our reproduction)

Origin. The cube part of KMCoders' Santa 2023 solution [19], implemented from their written description only, before we looked at their code. It targets the exact Santa goal and the Santa (QTM) metric.

The insertion identity. All later stages rest on one observation of KMCoders. For a move sequence $a_0 \dots a_{T-1}$ let suf_t be the permutation of the suffix $a_t \dots a_{T-1}$ and inv_t its inverse. Inserting a sequence B before move t turns the final state F into $F[C]$ with

$$C = \mathrm{inv}_t \circ B \circ \mathrm{suf}_t, \tag{3}$$

so a sequence that acts on a few stickers at time t acts on the stickers $\mathrm{inv}_t[\mathrm{support}(B)]$ of the final state. With all suffix permutations stored, scoring a candidate at every time is a gather over inv, with no replay; inserting where the neighbouring moves cancel the candidate's first or last move makes it shorter.

Steps.

(1) *Special parts.* Middle-slice turns bring odd cubes' fixed centers home (the Santa goal is exact). For even N the corners form a 2×2×2, solved optimally in QTM with Method 1's corner pattern database (this matches KMCoders' 2×2×2 lengths exactly); for odd N the corners, middle edges and fixed centers form a 3×3×3, solved by Kociemba's two-phase algorithm (KMCoders used an optimal solver).
(2) *Parity.* Every wing cluster with an odd permutation gets one inner-slice quarter turn.
(3) *Annealing.* Simulated annealing over the move sequence with objective length (QTM) + $\alpha \cdot$ #wrong stickers ($\alpha = 2.5$, $6000N^2$ iterations, temperature $2 \to 0.05$). Only inner, non-middle slice quarter turns may be edited, so special parts stay solved; neighbours insert two moves of one parity class at two times, remove two such moves, move one move to another time, change one move within its class, or swap two adjacent moves. Each keeps every cluster's permutation even and is scored exactly in $O(6N^2)$ with Eq. (3).
(4) *Commutators.* Greedy insertion of 4-move commutators $[x, y]$ of two slices (inner–inner, inner–face and face–inner), each a pair of 3-cycles: at each step the (commutator, time) pair with the best pieces fixed per move, net of cancellations, over 256 sampled times.
(5) *3-cycles.* Every cluster is finished with pure 3-cycles from a library that is QTM-optimal per 3-cycle (built by breadth-first search over conjugating setups), each inserted at the time where it is cheapest.

Cost. Storing suf_t and inv_t for all T times takes $O(T \cdot 6N^2)$ memory (about 21 MB at $N = 19$); an insertion rebuilds the suffixes before it in $O(t \cdot 6N^2)$. One greedy round scores $O(\text{times} \times \text{candidates})$ gathers. Every 3-cycle fixes $O(1)$ pieces, so solutions have $\Theta(N^2)$ moves. *What it leaves out.* Comparing with KMCoders' code shows what the write-up does not describe: their stage 3 also applies bulk commutators $[D_I, R_J]$ of whole slice sets I, J chosen by annealing (Demaine-style parallel moves), their final stage counts the exact minimum number of 3-cycles each cluster still needs and inserts only 3-cycles that lower it, with a beam of 10^5 sequences for small N, and they solve odd cubes' central 3×3×3 optimally.

Algorithm 7: Method 6: KMCoders pipeline (our reproduction from the write-up)

Input: state s (exact goal s^*)
Output: move sequence a (quarter turns)
$a \leftarrow$ orientation moves ‖ optimal special-part solution ‖ wing parity fixes;
$a \leftarrow \textsc{Anneal}(a)$ over parity-preserving inner-slice edits, objective $|a| + \alpha \cdot \mathrm{wrong}(s \cdot a)$;
while *many stickers wrong* **do**
  $(B, t) \leftarrow \arg\max \mathrm{gain}(B, t)/\mathrm{cost}(B, t)$ over 4-move commutators B, sampled times t ; // Eq. (3)
  insert B before move t; update suf, inv;
for *each cluster c until solved* **do**
  pick a 3-cycle B of c's library and the time t where it is cheapest; insert it;
return a;

7.8 Method 7: a symmetry portfolio around deterministic solvers (new, main method)

Idea. A deterministic solver that is far from optimal is not equally far on every input: KMCoders' program, run once on the same scramble seen from different sides, returns solutions whose lengths differ by 18–28 quarter turns on one 4×4×4 (Figure 3). The 48 symmetries of the cube turn one scramble into 48 equivalent inputs, and every answer maps back to a solution of the original state of the same length. Method 7 runs the solver on k copies and keeps the shortest. The same holds for the Rubik's Square, whose 4 rotations and their mirror images give 8 copies; there the inner solvers are our own.

Symmetric copies. Let R be a sticker permutation of one of the 24 rotations of the cube, or of one of their 24 mirror images (rotation composed with the reflection $x \mapsto -x$). Let σ be the colour relabelling with $\sigma(s^*[R]) = s^*$ for the solved state s^*. The copy of a state s is $s' = \sigma(s[R])$.

Proposition. *For every move q there is a move $\phi_R(q)$ of the same cost with $R \circ q \circ R^{-1} = \phi_R(q)$ (as sticker permutations); for a rotation ϕ_R maps a quarter turn of a slice to a quarter turn of the image slice, for a mirror image to the opposite quarter turn of the mirrored slice. Hence if $q_1 \dots q_L$ solves s' exactly, then $\phi_R(q_1) \dots \phi_R(q_L)$ solves s exactly, with the same QTM, HTM and STM.*

Proof sketch. A rotation or reflection maps every slab of the cube onto a slab, so conjugating a slab turn by R gives a turn of the image slab by the same angle, with the sense reversed for a reflection; the move set is closed under this map and costs are preserved. Applying the moves to s' is the gather $s'[q_1 \circ \dots \circ q_L]$; substituting $s' = \sigma(s[R])$ and $q_i = R^{-1}\phi_R(q_i)R$ telescopes the inner $R^{-1}R$ pairs, so the result is $\sigma(s[\phi_R(q_1) \dots \phi_R(q_L)][R])$, which equals s^* exactly when $s[\phi_R(q_1) \dots \phi_R(q_L)] = s^*$ (because $\sigma(s^*[R]) = s^*$ and R

Algorithm 8: Method 7: symmetry portfolio

Input: state s; inner solver $\mathcal{S}$; number of copies k
Output: solution of s

$\mathcal{R} \leftarrow$ (identity, mirror, R_1, mirror(R_1), …) truncated to k;
best $\leftarrow \bot$;
for each $R \in \mathcal{R}$ **in parallel do**
  $s' \leftarrow \sigma_R(s[R])$;
  $q \leftarrow \mathcal{S}(s')$; `// KMCoders' program, beam variant, short stage 2, optimal 3×3×3 stage`
  $\mu \leftarrow \phi_R(q)$;
  if $s \cdot \mu = s^*$ ***and*** $|\mu|_{\mathrm{QTM}} < |best|_{\mathrm{QTM}}$ **then** best $\leftarrow \mu$;
return best;

is a bijection). The harness verifies every mapped-back solution independently. □

Steps.

(1) *Inner solver.* KMCoders' program (B2) with three settings: their beam variant (`solve_cube_beam`), which their write-up recommends for small N; 1M instead of 20M annealing iterations in their second stage, which takes about three quarters of a 4×4×4 run and did not change lengths measurably at 0.5M, 2M or 20M iterations; and, for odd N, an optimal quarter-turn central 3×3×3 from `nxopt` (Kociemba's answer only defines the position; `nxopt` solves it optimally), supplied through their cache file. On Santa puzzle 210 this stage drops from 34 to 24 quarter turns.
(2) *Copies.* The identity, its mirror image, then the other rotations each followed by its mirror image, up to k (48 for the cube, 8 for the Square).
(3) *Square.* The inner solvers are the strongest we have at each size, and every copy is solved by each of them: Method 4's per-size network (W=4096) and Method 5C for $N \leq 8$, Method 5C and Method 3 above (Method 4 has no models beyond $N = 8$). The goal is any solved orientation, which the Proposition also preserves because the symmetries permute the solved orientations.
(4) *Portfolio.* Run the inner solver on each copy (three in parallel), map each answer back, verify it, and keep the shortest in QTM. For the 4×4×4 we also take the shortest over an 8-copy portfolio of the default program (greedy, 20M iterations).

Cost. k times the inner solver's CPU time; memory is the inner solver's (plus `nxopt`'s 1.42 GB table, shared). Measured: 17–28 CPU-minutes per 4×4×4 puzzle for 48 copies and 28–46 per 5×5×5 puzzle for 24 copies, on a shared 4-core machine. Moves: as KMCoders' method, $\Theta(N^2)$ ($\approx 4.4N^2$ in the published data); the portfolio lowers the constant, not the exponent.

What did not help. Two attempts to improve KMCoders' finished solutions directly found nothing: simulated annealing over the free moves of a finished solution (Method 6's annealer with a large α) removed no move on two 4×4×4 puzzles, and a ruin-and-recreate local search [31] inside their stage 3 (delete the inner-slice moves of a random window in parity-safe pairs, let their greedy repair it, accept if not longer) found no shorter solution in 21 rounds. Their solutions are locally tight; diversity of the whole run, not local repair, is what pays.

8 EXPERIMENTS

Setup. All runs used CPUs except the GPU training runs named in the text (Colab T4). Method 5A's and Method 4's Square evaluations ran on a 2-core machine or on the T4; Methods 1–3, Method 5C, Method 6, Method 7 and the external baselines on a 4-core, 15 GB machine, often several jobs at once, so times are comparable within a table only. Every method sees the same instances (§4.1), and every number below is generated by script from the harness's result files. Means are over solved instances; a superscript gives the solved fraction when it is below the full count.

We first give the two main tables (§8.1), one per puzzle, with every method and baseline on the same instances; then check our reproductions against their originals (§8.2), analyse our methods (§8.3) and ask how far everything is from optimal (§8.4).

8.1 Main results

Tables 4 and 5 are the paper's main results. On the **Square**, Method 7, the symmetry portfolio over the 8 rotations and reflections of each state with the strongest solvers we have as inner solvers (Method 4 and Method 5C for $N \leq 8$, Method 5C and Method 3 above), is the best method at 18 of the 18 sizes. Where the optimum is known ($N \leq 5$) it reaches it, like Method 4's per-size networks; above $N = 8$, where no per-size network exists, it improves on the best single solver, Method 5C, which is in turn shorter than our reproductions of Demaine's algorithm (Method 3) and of reduction (Method 2). Method 5A, one network for every size, is the strongest method that needs no per-size training up to $N = 7$. On the **Cube**, Method 7 is the best method at 3 of the 8 Santa sizes (5, 6, 7); at every other size KMCoders' published solutions are best and one run of their program is second. Every other method, ours and the external solver dwalton76, is far behind on Santa 2023: these puzzles demand the exact goal and the quarter-turn metric, for which KMCoders' insertion pipeline was built.

8.2 Reproductions against their originals

Method 1 and exact optima. Breadth-first search from all four solved orientations visits all 768, 82,944 and 10,616,832 states of the 3×3×1, 4×4×1 and 5×5×1; their diameters are 6, 10 and 12 moves. The 3×3×1 (the "Floppy Cube") values agree with the published ones for the same move set; for the 4×4×1 and 5×5×1 we found no published value. IDA* solves random Squares optimally as follows: N=3: 10/10, N=4: 10/10, N=5: 10/10, N=6: 0/10 (120 s per state; 13 s per 5×5×1 on average). At $N = 6$ the max-cluster heuristic is too weak and every search ran out of time. On the 3×3×3, the pattern databases have exactly the sizes of Korf's [23]; on 100 random scrambles of 12 moves IDA* returned an optimal solution for 91/100 within 120 s each (13.2 HTM on average, optima between 8 and 16). On 2 random states it found no solution within 33 minutes each, after about $2.7 \cdot 10^{10}$ nodes, proving only that the optimum exceeds 16 moves. Korf reported optimal random 3×3×3 solutions with days of CPU time per state in 1997; our reproduction is correct but not fast enough for that benchmark on four cores.

Table 4: Rubik's Square, random states: mean solution length (HTM; on the Square HTM = QTM) of every method on the same instances (n per size). In each row the best method is red and the second best green; only methods that solved every instance of that size are ranked, and ties share the colour. Method 1: exact optimum (breadth-first search for $N \le 4$, IDA* for $N = 5$). Method 4: per-size network, W=4096. A superscript gives the solved fraction when below 100%. A dash means no result, for three reasons: Method 1 at $N \ge 6$, where no optimal solver finishes (0/10 states at $N = 6$); Method 4 at $N \ge 9$, for which no network was trained; Method 5A at $N = 3$, 13 and 15–20, not run.

N	n	Method 7 (ours)	Method 1 optimal	Method 4	Method 5A (ours)	Method 5C (ours)	Method 3	Method 2
3	10	**3.6**	**3.6**	**3.6**	–	**4.2**	**4.2**	**4.2**
4	10	**6.4**	**6.4**	**6.4**	**6.4**	**10.4**	**10.4**	**10.4**
5	10	**8.8**	**8.8**	**8.8**	**9.1**	20.7	20.7	20.7
6	10	**13.0**	–	**13.0**	**13.2**	28.8	28.8	30.6
7	10	**15.0**	–	**15.0**	**16.6**	37.9	37.9	39.9
8	10	**18.8**	–	**18.8**	54.2	**53.8**	54.4	58.0
9	10	**62.4**	–	–	68.0	**65.0**	65.2	68.4
10	10	**78.6**	–	–	87.8	**84.8**	85.6	94.4
11	10	**91.0**	–	–	96.6	**92.8**	93.0	103.2
12	10	**107.0**	–	–	$125.1^{9/10}$	**112.8**	113.4	132.0
13	5	**127.2**	–	–	–	**130.8**	132.4	148.0
14	5	**148.4**	–	–	187.6	**155.2**	156.4	180.4
15	5	**165.4**	–	–	–	**167.4**	169.0	194.6
16	5	**189.6**	–	–	–	**194.8**	196.8	236.0
17	5	**203.0**	–	–	–	**206.6**	209.0	247.4
18	5	**223.2**	–	–	–	**236.8**	240.0	292.4
19	5	**259.0**	–	–	–	**262.2**	271.0	327.8
20	5	**276.0**	–	–	–	**295.6**	304.0	379.6

Table 5: Rubik's Cube, Santa 2023 puzzles: mean solution length in QTM, the competition's score, of every method on the same puzzles (n per size). In each row the best method is red and the second best green; only methods that solved every puzzle of that size are ranked. B1: KMCoders' published best-of-many solutions; B2: one run of their program; B3: dwalton76 (§6). A superscript gives the number of puzzles run when below n. A dash means not run, because of cost: Method 7 and B2 at $N = 19$ (hours of CPU per copy); B3 at $N \ge 8$ (run for $N \le 7$ only).

N	n	Method 7 (ours)	B1 KMCoders published	B2 KMCoders program, one run	B3 dwalton76	Method 6	Method 5C (ours)	Method 3	Method 2
4	43	$63.1^{15/43}$	**66.7**	**77.0**	88.6	109.4	217.5	229.3	266.9
5	19	**95.0**	**97.1**	114.2	160.2	159.3	282.5	296.3	361.1
6	6	**158.3**	**161.0**	179.3	324.0	276.0	503.0	534.7	696.3
7	4	**205.2**	**208.8**	232.8	539.0	356.2	593.8	680.8	880.2
8	3	–	**291.3**	**310.7**	–	468.7	856.0	993.3	1,297.3
9	3	–	**354.7**	**390.0**	–	572.7	1,020.0	1,188.0	1,594.0
10	4	–	**461.5**	–	–	**743.0**	1,314.5	1,597.0	2,107.5
19	3	–	**1,620.0**	–	–	**2,484.7**	4,710.7	6,300.7	8,188.7

Method 4 against CayleyPy. On the Square, our per-size networks behave as CayleyPy reports for cubes: they found the optimum on every 3×3×1 and 4×4×1 instance (10/10 and 10/10, Table 6), matched Method 1's optimal mean on the 5×5×1, and wider beams help (Figure 2): on the 8×8×1, Method 4 solved 3/10 states at W=16, 9/10 at W=256 and all from W=1024 on, at 17.4 s per state for W=4096. We did not train cube models at CayleyPy's scale (8 billion examples); Method 4 is therefore a Square method in this paper.

Method 4 with GPU-trained models. A second set of Square models, trained on a Colab T4 with the "large" preset (1.27M parameters and 256M examples at $N = 8$, about 7 minutes per model), gives the same lengths as the CPU-trained models up to $N = 6$ and is better only at $N = 8$ (18.8 against 20.3 HTM, shorter on 12 of 20 states and longer on none; Table 7). The larger model is the default at $N = 8$ only.

Method 6 and one run of KMCoders' program against KMCoders' published lengths. Table 8 compares the two ways of reproducing KMCoders on the Santa puzzles. Our reproduction from the write-up (Method 6) is 1.5–1.7× their published lengths at every size, but it is the best of our non-KMCoders methods from $N = 6$ on (Table 5). One run of their own program is only 7–18% longer than the published lengths. The gap between Method 6 and B2 is what the write-up leaves out (§7.7); the gap between B2 and B1 is what

Table 6: Optimality on the smallest Squares. "Gap" = mean HTM above the exact optimum over solved instances; "optimal" = instances solved in exactly the optimal length. Method 5A was evaluated for $N \geq 4$ only.

Puzzle	God's	mean opt.	Method	solved	gap	optimal
3×3	6	3.6	Method 4 (W=4096)	10/10	0.0	10/10
			Method 5A	–	–	–
			Method 5B	10/10	1.2	1/10
			Methods 2, 3, 5C	10/10	0.6	5/10
4×4	10	6.4	Method 4 (W=4096)	10/10	0.0	10/10
			Method 5A	10/10	0.0	10/10
			Method 5B	10/10	7.0	0/10
			Methods 2, 3, 5C	10/10	4.0	3/10

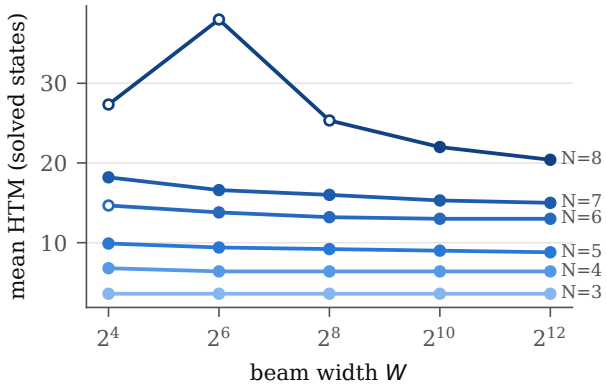


Figure 2: Method 4: mean solution length against beam width W, one per-size network for each Square size $N = 3, \ldots, 8$ (10 states each; means over solved states). Hollow markers: not all 10 states solved.

Table 7: Method 4 on the same 20 random Squares per size, same CPU, W=4096: committed CPU-trained models against the GPU-trained "large" models. Last column: states with a shorter / longer solution with the GPU model. Every state was solved by both.

N	n	optimal	CPU-trained	GPU-trained	shorter / longer
3	20	3.75	3.75	3.75	0 / 0
4	20	6.70	6.70	6.70	0 / 0
5	20	–	9.10	9.10	0 / 0
6	20	–	12.60	12.60	0 / 0
7	20	–	15.40	15.60	0 / 4
8	20	–	20.30	18.80	12 / 0

many runs and tuning bought in the competition, and it is what Method 7 goes after.

Table 8: Reproducing KMCoders on the Santa 2023 cubes (mean QTM). B1: their published solutions; B2: one run of their program (puzzles run in brackets); Method 6: our reproduction from their write-up. Ratios to B1.

N	B1 published	B2 one run	B2 / B1	Method 6	Method 6/ B1
4	66.7	77.0 (43)	1.15	109.4	1.64
5	97.1	114.2 (19)	1.18	159.3	1.64
6	161.0	179.3 (6)	1.11	276.0	1.71
7	208.8	232.8 (4)	1.11	356.2	1.71
8	291.3	310.7 (3)	1.07	468.7	1.61
9	354.7	390.0 (3)	1.10	572.7	1.61

8.3 Analysis of our methods

Method 7 against every baseline, puzzle by puzzle. Table 9 compares Method 7 with every baseline and every other method on the same puzzles. It is the shortest method at every size it has run: on the 44 puzzles, its total is 4,522 quarter turns against 4,620 for KMCoders' published solutions (2.1% fewer), shorter on 23, equal on 10 and longer on 11; one run of their program, dwalton76 and all our other methods are further behind.

Why the portfolio works. Figure 3 shows the 48 copies of each of the first ten 4×4×4 puzzles: they spread over 18–28 quarter turns (all copies: 60 to 89, mean 75.2), and the published solution sits in the lower tail of that spread. Figure 4 shows the best of k random copies: the mean falls quickly at first and is still falling at $k = 48$. For odd N the optimal central 3×3×3 (`nxopt`) matters as much as the portfolio: it saves about 10 quarter turns per copy on the 5×5×5 (34 to 24 on puzzle 210); with Kociemba's stage the same 24-copy portfolio averaged 102.0 on the first three 5×5×5 puzzles, against 95.3 with `nxopt` and 100.7 published.

Time. Table 10 gives the mean wall time per Santa puzzle. One run of KMCoders' program takes about a minute at $N = 4$ and 22 minutes at $N = 9$ on our shared machine, so Method 7's k copies cost k times that; our cluster methods take seconds to minutes even at $N = 19$.

Statistics with 20 states per size. Table 11 repeats the cube comparison of Methods 2, 3 and Method 5C with 20 states per size and 95% confidence intervals, and gives the paired difference Method 5C −Method 3 on the same states. Method 5C is significantly shorter than Method 3 from $N = 5$ on, and the gap grows steadily with N; at $N = 4$, with one center and one wing cluster and nothing to batch, the difference is not significant.

Where Method 5C's gain comes from. Method 5C's advantage over Method 3 grows with N: 6% at the 6×6×6, 21% at the 10×10×10, 23% at the 15×15×15 and 26% at the 20×20×20. The gain is in the centers, which are 85% of Method 5C's moves on the 20×20×20: there, 229 of its 333 center rounds were bulk moves, against Method 3, where 244 of 292 rounds fixed a single cluster. Table 12 removes the lookahead and the setup sharing. Net-gain blocks alone already give most of the gain (on the 20×20×20, 4,925 moves against Method 3's 6,612). One step of lookahead shortens solutions by a further 2–8% (4,794 moves on the 20×20×20). Setup sharing matters least.

Table 9: Method 7 against every baseline and every other method, on exactly the Santa 2023 puzzles Method 7 was run on (mean QTM, the competition's score; bold: shortest). B1: KMCoders' published best-of-many solutions; B2: one run of their program; B3: dwalton76. W/T/L: puzzles where Method 7 is shorter than / equal to / longer than B1; last column: Method 7's saving over B1. Last row: total QTM over all these puzzles, i.e. the Santa score restricted to them.

	run	Method 7 (ours)	B1 published	B2 one run	B3 dwalton76	Method 6	Method 5C	Method 3	Method 2	W/T/L vs B1	saving
4×4×4	15/43	**63.1**	64.9	77.5	88.7	108.8	216.3	226.0	263.5	6/5/4	+2.9%
5×5×5	19/19	**95.0**	97.1	114.2	160.2	159.3	282.5	296.3	361.1	11/3/5	+2.2%
6×6×6	6/6	**158.3**	161.0	179.3	324.0	276.0	503.0	534.7	696.3	4/2/0	+1.7%
7×7×7	4/4	**205.2**	208.8	232.8	539.0	356.2	593.8	680.8	880.2	2/0/2	+1.7%
all	44	**4,522**	4,620	(total QTM, the Santa score on these puzzles)						23/10/11	+2.1%

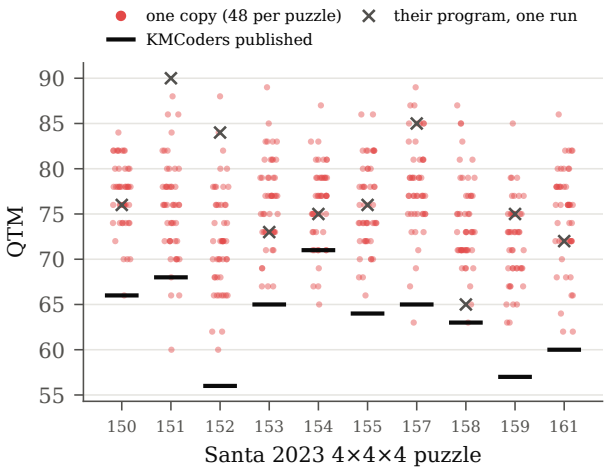


Figure 3: Method 7 on the first 10 Santa 4×4×4 puzzles: QTM of each of the 48 symmetric copies (dots), KMCoders' published solution (bar) and one run of their program on the unrotated puzzle (cross).

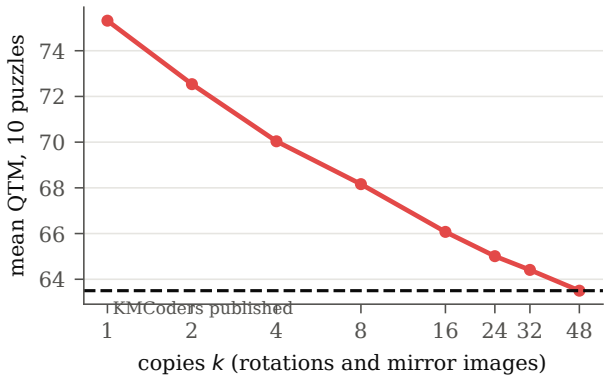


Figure 4: Best of k symmetric copies, averaged over the 10 4×4×4 puzzles (and 200 random draws of the k copies), against KMCoders' published mean (dashed).

Table 10: Mean wall time per Santa puzzle (seconds; shared 4-core machine, so indicative).

N	B2 one run	B3	Method 6	Method 5C	Method 3	Method 2
4	64	8	6	0	0	0
5	131	22	10	1	0	0
6	250	35	16	3	0	1
7	473	38	23	6	0	2
8	824	–	36	11	1	4
9	1,308	–	46	17	1	4
10	–	–	67	26	2	4
19	–	–	850	88	27	26

Table 11: Cube, 20 random states per size: mean HTM ± 95% confidence half-width, and the paired difference Method 5C −Method 3 on the same states (*: interval excludes 0).

N	Method 2	Method 3	Method 5C	Method 5C −Method 3	n
3	20.7 ± 0.4	20.7 ± 0.4	20.7 ± 0.4	+0.0 ± 0.0 (+0.0%)	20
4	212.6 ± 4.5	186.8 ± 5.0	184.1 ± 4.1	-2.8 ± 3.4 (-1.5%)	20
5	321.2 ± 6.0	262.9 ± 5.4	253.3 ± 5.8	-9.7 ± 4.8 (-3.7%)*	20
6	589.0 ± 9.5	470.7 ± 6.2	434.6 ± 9.3	-36.1 ± 8.7 (-7.7%)*	20
7	803.6 ± 10.7	632.6 ± 12.2	572.0 ± 8.7	-60.6 ± 10.6 (-9.6%)*	20
8	1,144.5 ± 13.6	902.5 ± 13.3	795.6 ± 7.9	-106.9 ± 13.1 (-11.8%)*	20
9	1,453.2 ± 17.5	1,153.3 ± 15.9	965.1 ± 10.7	-188.2 ± 17.3 (-16.3%)*	20
10	1,885.3 ± 16.2	1,489.8 ± 15.7	1,234.2 ± 13.1	-255.6 ± 18.6 (-17.2%)*	20

Method 5A: does one network generalise across sizes? Method 5A was trained twice on Squares with $N \in \{4, \ldots, 8\}$ only: a small CPU model (198,658 parameters, 2,500 steps) and a larger GPU model (width 192, four layers, six heads, 20,000 steps of 512 examples on a Colab T4), which is the one in Table 4. Table 13 shows that the CPU model's distance estimate does not collapse on sizes it never saw: its correlation with the true walk depth is 0.64 at $N = 8$, 0.66 at $N = 10$ and 0.53 at $N = 20$; the GPU model keeps 0.70–0.80 on $N = 10$, 14 and 20 at the end of training. As a solver the GPU model solved every Square we gave it; it is second only to Method 4's per-size networks up to $N = 7$, but above that the hand-designed schedules of Method 5C and Method 3 are shorter. A cube model

Table 12: Method 5C components on the cube (mean HTM, same instances): Method 3's schedule; Method 5C without lookahead; full Method 5C (lookahead on, the default for $N \leq 15$); Method 5C with lookahead but without setup sharing. "–" = not run.

N	Method 3	no lookahead	with lookahead	no setup sharing
6	464.5	450.5	437.0	436.5
7	633.0	578.0	564.0	570.5
8	897.5	836.0	806.5	810.5
10	1,509.0	1,296.0	1,196.5	1,246.0
15	3,578.0	2,836.0	2,765.0	–
20	6,612.0	4,925.0	4,794.0	–

Table 13: Method 5A, trained on Squares with $N \in \{4, \ldots, 8\}$ only: agreement of its distance estimate with the true random-walk depth on fresh walks (* = size never seen in training). RMSE is in units of $N^2/\ln N$.

N	4	5	6	7	8	10*	14*	20*
Pearson r	0.48	0.62	0.67	0.61	0.64	0.66	0.65	0.53
normalised RMSE	0.71	0.63	0.58	0.61	0.57	0.57	0.60	0.70

trained the same way (50,000 steps, $N = 3$–7) solved none of 10 random cubes, so Method 5A is a Square method in this paper.

Scaling with N. Figure 5 divides each mean length by $N^2/\ln N$. Every method's ratio still rises over the tested range, so none is shown to grow at the $N^2/\log N$ rate that Demaine's construction achieves asymptotically; Method 5C's rises most slowly.

8.4 How far from optimal?

Figure 6 divides each method's mean Santa length by the counting bound of §4.2. KMCoders' published solutions are 1.81× the bound at $N = 4$, 2.22× at $N = 10$, 2.40× at $N = 19$ and 2.54× at $N = 33$. On the Square, where we know the truth, the worst case is 1.5–1.7× the bound (Table 2), and typical optimal distances are lower still, so at small N the published solutions are within a small factor of optimal, and the room left grows with N. The growth has a clear cause: the published lengths are a near-constant 4.62, 4.49 and $4.33 \cdot N^2$ at $N = 10$, 19 and 33, while the bound grows like $N^2/\log N$. KMCoders' final stage fixes $O(1)$ pieces per 3-cycle, so it is $\Theta(N^2)$, although their stage 3 already contains Demaine-style bulk commutators. A method that kept the parallelism of bulk commutators into the endgame, as Method 5C does on the random benchmark, is the natural way to close this gap at large N; on Santa today, Method 5C's constant is still too large.

9 LIMITATIONS

- *Method 7's evidence is small and its cost is high.* We ran Method 7 on 10 of the 43 Santa 4×4×4 puzzles and 3 of the 19 5×5×5 puzzles, at about half an hour of CPU per puzzle. It is shorter than the published solution on all three 5×5×5 puzzles but only on average, not on every puzzle, at $N = 4$; with so few puzzles neither result is a significant difference. We have not run it at $N \geq 6$ (each copy takes 4 minutes at $N = 6$ and 22 at $N = 9$ on our machine), nor on the 19×19×19 and 33×33×33. We have not yet compared it with an equal-compute portfolio of random seeds of the same program; the seed also changes the answer, and symmetries are one convenient, seed-free source of the same diversity.
- *Method 7 builds on KMCoders' code.* Its contribution is the portfolio, the shortened stage and the optimal central stage around their solver; the algorithmic core is theirs (§7.8). Their repository carries no licence file, so we release only a wrapper and a two-line patch, not their code.
- *Random-state samples.* The main tables use few states per size for large cubes (one each at $N = 15$ and 20); the 20-state confidence intervals cover $N \leq 10$.
- *Still above Demaine's rate.* Every method's length divided by $N^2/\ln N$ still rises over the tested range, Method 5C's most slowly; KMCoders' lengths grow as $\approx 4.4N^2$. None is shown to reach the $\Theta(N^2/\log N)$ rate in practice.
- *The counting bound is a lower bound.* It is valid for the radius around the goal, and we use it as an estimate of typical optimal distances; the Square calibration suggests the truth is 1.5–1.7× the bound in the worst case. It cannot certify that any single solution is near-optimal.
- *Exact search on the 3×3×3* solves 12-move scrambles but no random state in 30 minutes on four cores. We did not train cube networks at CayleyPy's scale, so Method 4 and Method 5A are evaluated on the Square only.
- *Checker and sampling.* The checker enforces necessary conditions only, and random states come from long random walks rather than an exact uniform sampler.

10 CONCLUSION

Decomposing Rubik's puzzles into clusters with N-invariant signatures lets one engine run eight solvers on the same instances, from exact search to learned heuristics and a competition solver, for every size we can afford. On random states the winner at large N is structural: Demaine's idea of fixing many clusters with one shared move sequence, scheduled by net gain rather than by unanimous need (Method 5C). On the Santa 2023 puzzles, the competition's own solver is far ahead of everything else, and the symmetries of the cube are enough to turn it into a portfolio that reaches its published best-of-many lengths (Method 7). The counting bound says where the remaining room is: little at $N = 4$, and a factor that grows like $\log N$ at large N, because the final stage of every practical method still fixes a few pieces per 3-cycle. Carrying Method 5C's bulk moves into the endgame of the insertion method is the natural next step, and the place where the two lines of this work meet. Code, data and result files are at https://github.com/gauravrai1704/Rubik.

ACKNOWLEDGMENTS

This work was carried out as a course project at Shiv Nadar University.

REFERENCES

[1] Forest Agostinelli, Stephen McAleer, Alexander Shmakov, and Pierre Baldi. 2019. Solving the Rubik's Cube with Deep Reinforcement Learning and Search. *Nature*

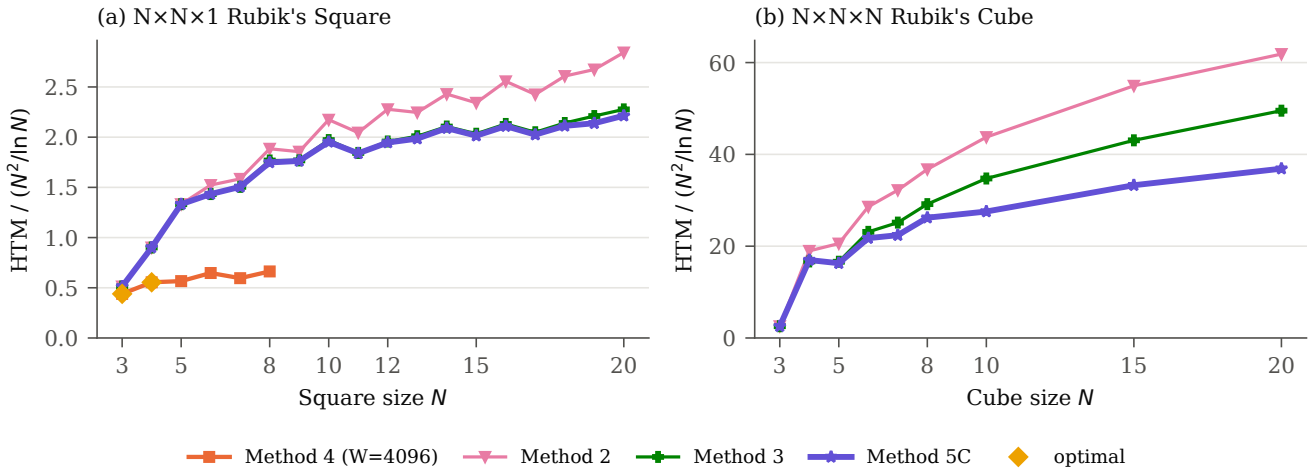


Figure 5: Solution length divided by $N^2/\ln N$, the growth rate of the worst case, for the Square (a) and the cube (b), random states. A flat line would mean growth at the rate of the worst case. Means over solved states.

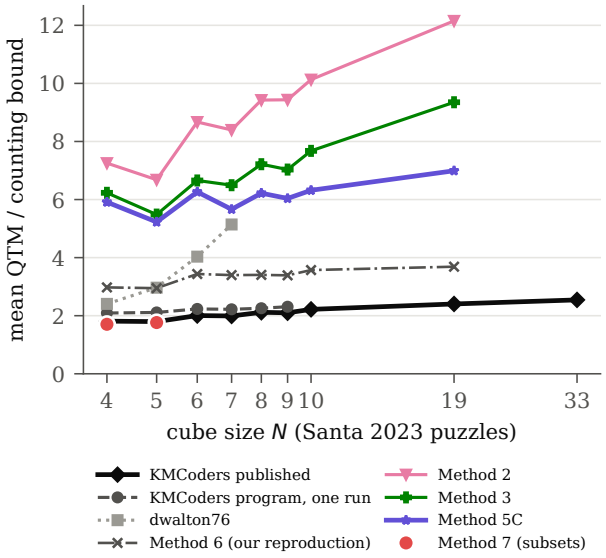


Figure 6: Mean Santa 2023 length divided by the counting lower bound, against N (log scale). Methods shown only where they ran on every puzzle of that size; Method 7 on the puzzles of Table 9.

Machine Intelligence 1, 8 (2019), 356–363. https://doi.org/10.1038/s42256-019-0070-z

[2] Forest Agostinelli, Shahaf S. Shperberg, Alexander Shmakov, Stephen McAleer, Roy Fox, and Pierre Baldi. 2024. A* Search Without Expansions: Learning Heuristic Functions with Deep Q-Networks. arXiv:2102.04518 [cs.AI]

[3] Dhruv Anand and Ehsan Shareghi. 2025. Cube Bench: A Benchmark for Spatial Visual Reasoning in MLLMs. arXiv:2512.20595

[4] Huan ang Gao, Zikang Zhang, Tianwei Luo, Kaisen Yang, Xinzhe Juan, Jiahao Qiu, Tianxing Chen, Bingxiang He, Hao Zhao, Hao Zhou, Shilong Liu, and Mengdi Wang. 2025. CubeBench: Diagnosing Interactive, Long-Horizon Spatial Reasoning Under Partial Observations. arXiv:2512.23328

[5] Shuang Chen. 2026. min2phase: An Implementation of Kociemba's Two-Phase Algorithm. https://github.com/cs0x7f/min2phase. Accessed October 2026.

[6] Alexander Chervov, Dmytro Fedoriaka, Mark Obozov, Elena Konstantinova, et al. 2025. CayleyPy Growth: Efficient Growth Computations and Hundreds of New Conjectures on Cayley Graphs. arXiv:2509.19162

[7] Alexander Chervov, Kirill Khoruzhii, Nikita Bukhal, Jalal Naghiyev, Vladislav Zamkovoy, Ivan Koltsov, Lyudmila Cheldieva, Arsenii Sychev, Arsenii Lenin, Mark Obozov, Egor Urvanov, and Alexey Romanov. 2025. A Machine Learning Approach That Beats Rubik's Cubes. In *Advances in Neural Information Processing Systems (NeurIPS)*. Spotlight. arXiv:2502.13266.

[8] Joseph C. Culberson and Jonathan Schaeffer. 1998. Pattern Databases. *Computational Intelligence* 14, 3 (1998), 318–334.

[9] Erik D. Demaine, Martin L. Demaine, Sarah Eisenstat, Anna Lubiw, and Andrew Winslow. 2011. Algorithms for Solving Rubik's Cubes. In *Proceedings of the 19th Annual European Symposium on Algorithms (ESA 2011) (Lecture Notes in Computer Science, Vol. 6942)*. Springer, 689–700. Full version: arXiv:1106.5736.

[10] Erik D. Demaine, Sarah Eisenstat, and Mikhail Rudoy. 2018. Solving the Rubik's Cube Optimally is NP-complete. In *35th Symposium on Theoretical Aspects of Computer Science (STACS 2018) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 96)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 24:1–24:13. https://doi.org/10.4230/LIPIcs.STACS.2018.24

[11] Jordan Ellenberg. 2011. Should You Be Surprised by the Diameter of the $n \times n \times n$ Rubik's Group? Blog post, Quomodocumque, https://quomodocumque.wordpress.com/2011/07/17/should-you-be-surprised-by-the-diameter-of-the-nxnxn-rubiks-group/.

[12] Gaurav, Shree, Sunaina, Kashvi, Shaurya, and Sharanya. 2026. Rubik: Nine Solvers for N×N×1 and N×N×N Rubik's Puzzles on One Engine. https://github.com/gauravrai1704/Rubik.

[13] Carla P. Gomes and Bart Selman. 2001. Algorithm Portfolios. *Artificial Intelligence* 126, 1–2 (2001), 43–62.

[14] Carla P. Gomes, Bart Selman, and Henry Kautz. 1998. Boosting Combinatorial Search Through Randomization. In *Proceedings of the Fifteenth National Conference on Artificial Intelligence (AAAI-98)*. 431–437.

[15] Bernardo A. Huberman, Rajan M. Lukose, and Tad Hogg. 1997. An Economics Approach to Hard Computational Problems. *Science* 275, 5296 (1997), 51–54.

[16] David Joyner. 2008. *Adventures in Group Theory: Rubik's Cube, Merlin's Machine, and Other Mathematical Toys* (2 ed.). Johns Hopkins University Press.

[17] Kaggle. 2023. Santa 2023 – The Polytope Permutation Puzzle. https://www.kaggle.com/competitions/santa-2023.

[18] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. 1983. Optimization by Simulated Annealing. *Science* 220, 4598 (1983), 671–680.

[19] KMCoders. 2024. Kaggle Santa 2023 – The Polytope Permutation Puzzle: KMCoders' Solution. https://github.com/wata-orz/santa2023_permutation_puzzle.

[20] Herbert Kociemba. 2026. Cube Explorer and Optimal Solvers for the Rubik's Cube. http://kociemba.org/download.htm. Accessed November 2026.

[21] Herbert Kociemba. 2026. The Two-Phase Algorithm. https://kociemba.org/cube.htm. Accessed October 2026.

[22] Richard E. Korf. 1985. Macro-Operators: A Weak Method for Learning. *Artificial Intelligence* 26, 1 (1985), 35–77.

[23] Richard E. Korf. 1997. Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*. 700–705.

[24] Daniel Kunkle and Gene Cooperman. 2007. Twenty-Six Moves Suffice for Rubik's Cube. In *Proceedings of the 2007 International Symposium on Symbolic and Algebraic Computation (ISSAC)*. 235–242.

[25] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. 2019. Set Transformer: A Framework for Attention-Based Permutation-Invariant Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.

[26] Bruce T. Lowerre. 1976. *The HARPY Speech Recognition System*. Ph. D. Dissertation. Carnegie Mellon University.

[27] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. 2019. Solving Rubik's Cube with a Robot Hand. arXiv:1910.07113

[28] Tomas Rokicki. 2026. cube20src: nxopt, an Optimal Solver for the Rubik's Cube in the Half- and Quarter-Turn Metrics. https://github.com/rokicki/cube20src. GPL-3.0; accessed November 2026.

[29] Tomas Rokicki and Morley Davidson. 2014. God's Number is 26 in the Quarter-Turn Metric. https://www.cube20.org/qtm/.

[30] Tomas Rokicki, Herbert Kociemba, Morley Davidson, and John Dethridge. 2014. The Diameter of the Rubik's Cube Group Is Twenty. *SIAM Rev.* 56, 4 (2014), 645–670. https://doi.org/10.1137/14097375X

[31] Paul Shaw. 1998. Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems. In *Principles and Practice of Constraint Programming (CP 98) (Lecture Notes in Computer Science, Vol. 1520)*. Springer, 417–431.

[32] David Singmaster. 1981. *Notes on Rubik's Magic Cube*. Enslow Publishers.

[33] Kyo Takano. 2023. Self-Supervision Is All You Need for Solving Rubik's Cube. *Transactions on Machine Learning Research* (2023). arXiv:2106.03157.

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[35] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. 2015. Pointer Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[36] Daniel Walton. 2026. rubiks-cube-NxNxN-solver. https://github.com/dwalton76/rubiks-cube-NxNxN-solver. Accessed October 2026.